

WEB APPLICATION DEVELOPMENT WITH NODE.JS

GV: TRẦN THANH TUẤN

03. ASYNCHRONOUS JAVASCRIPT

synchronous, single thread of control



synchronous, two threads of control



asynchronous



Example

```
function sayHello(name) {  
    console.log("Xin chào, " + name);  
}
```

```
function done() {  
    console.log("Hoàn tất!");  
}
```

```
function doAnotherTask() {  
    console.log("Làm việc khác sau khi chào!");  
}
```

```
sayHello("Tuan Tran");  
done();
```

```
sayHello("Tuan Tran");  
doAnotherTask();
```


Example

```
function sayHello(name) {  
  console.log("Xin chào, " + name);  
  done();  
}
```

```
sayHello("Tuan Tran");
```

```
function sayHello(name) {  
  console.log("Xin chào, " + name);  
  doAnotherTask();  
}
```

```
sayHello("Tuan Tran");
```

Other Example

```
function cong(a, b) {  
    console.log(`${a} + ${b} = ${a + b}`);  
}
```

```
function tru(a, b) {  
    console.log(`${a} - ${b} = ${a - b}`);  
}
```

```
function tinhToan(a, b) {  
    cong(a, b);  
}
```

```
function tinhToan(a, b) {  
    tru(a, b);  
}
```

```
tinhToan(5, 9);
```

```
tinhToan(9, 3);
```

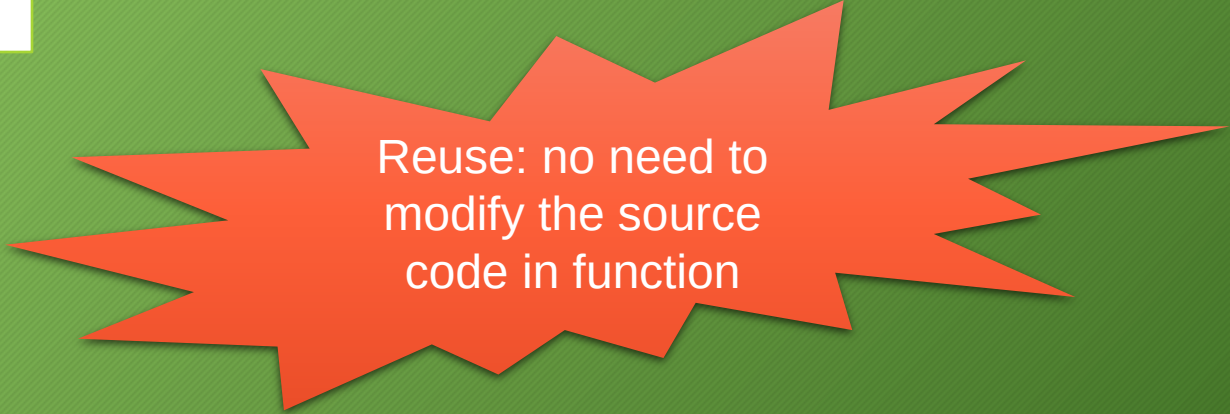
The problem

```
function sayHello(name) {  
  console.log("Xin chào, " + name);  
  done();  
}
```

```
function sayHello(name) {  
  console.log("Xin chào, " + name);  
  doAnotherTask();  
}
```

```
function tinhToan(a, b) {  
  cong(a, b);  
}
```

```
function tinhToan(a, b) {  
  tru(a, b);  
}
```



Reuse: no need to
modify the source
code in function

Solve the problem

```
function sayHello(name, callback) {  
  console.log("Xin chào, " + name);  
  callback();  
}
```

```
sayHello("Tuan Tran", done);
```

```
sayHello("Tuan Tran", doAnotherTask);
```

```
function tinhToan(a, b, callback) {  
  callback(a, b);  
}
```

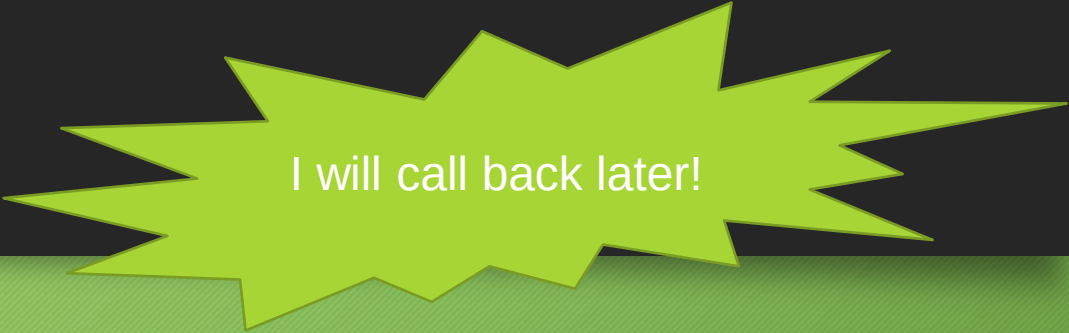
```
tinhToan(5, 9, cong);
```

```
tinhToan(5, 9, tru);
```



Use **callback** function

JavaScript Callbacks



I will call back later!

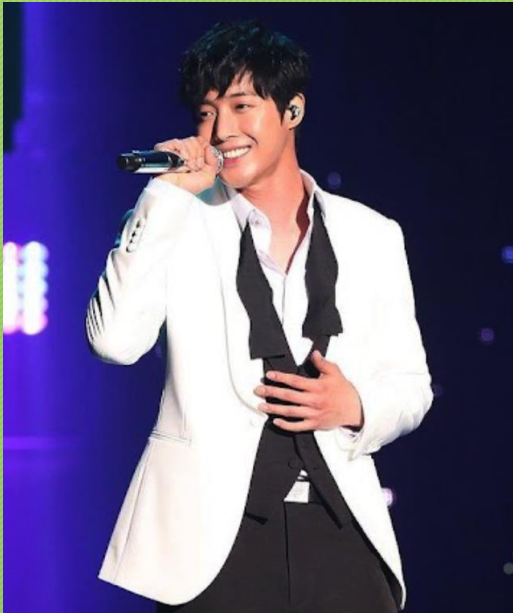
- A callback is a function passed as an argument to another function
- This technique allows a function to call another function
- A callback function can run after another function has finished

Callbacks are most often used with asynchronous functions.

```
setTimeout(sayHello, 3000);  
  
function sayHello() {  
    console.log("Hello world!");  
}
```

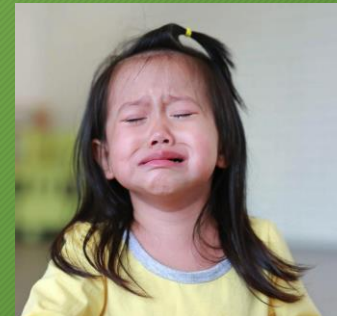
```
setTimeout(function() {  
    sayHello("world");  
}, 3000);  
  
function sayHello(value) {  
    console.log(`Hello ${value}`);  
}
```


JavaScript Promises



ask & fill the email to a list

promise send new song to them when it's published



JavaScript Promises

I Promise a Result!

does something and takes time

Producing code

Promise

Consuming code

wants the result of the **producing code** once it's ready

state

pending

fulfilled

rejected

result

undefined

result value

error value

JavaScript Promises

- Syntax

```
let thePromise = new Promise(function(resolve, reject) {  
    // "Producing Code" (May take some time)  
  
    resolve(result); // when successful  
    reject(error); // when error  
});
```

```
// "Consuming Code" (Must wait for a fulfilled Promise)  
thePromise.then(  
    function(value) { /* code if successful */ },  
    function(error) { /* code if some error */ }  
);
```

- Cannot access the Promise properties state and result.
- Must use a Promise method to handle promises.

JavaScript Promises

- Example

```
function printResult(result) {  
    console.log(result);  
}
```

```
let myPromise1 = new Promise(function(resolve, reject) {  
    let x = 10;  
    if (x !== 0) {  
        resolve("OK");  
    } else {  
        reject("Error");  
    }  
});
```

```
myPromise1.then(  
    function(value) {  
        printResult(value);  
    },  
    function(error) {  
        printResult(error);  
    }  
);
```

OK

```
let myPromise2 = new Promise(function(resolve, reject) {  
    let x = 0;  
    if (x !== 0) {  
        resolve("OK");  
    } else {  
        reject("Error");  
    }  
});
```

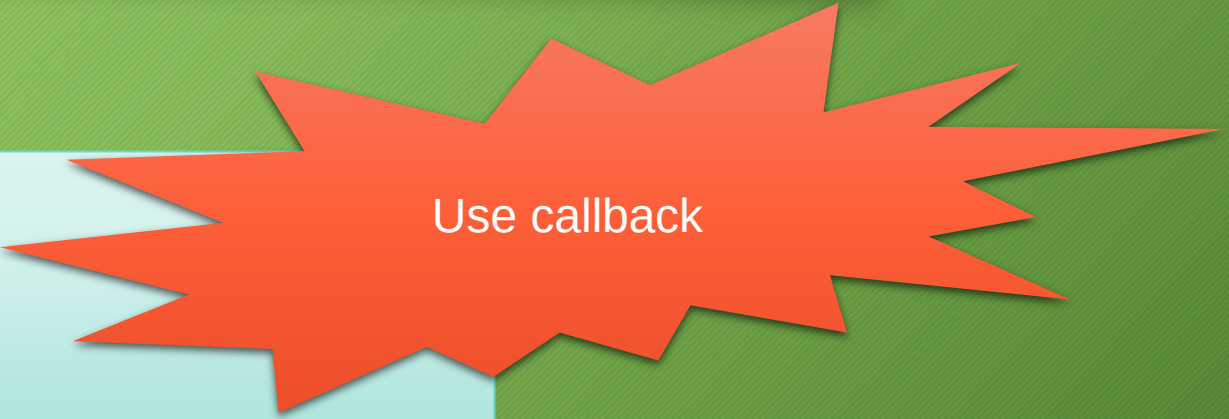
```
myPromise2.then(  
    function(value) {  
        printResult(value);  
    },  
    function(error) {  
        printResult(error);  
    }  
);
```

Error

JavaScript Promises

- Example

```
setTimeout(function() {  
    sayHello("world!");  
}, 3000);  
  
function sayHello(value) {  
    console.log(`Hello ${value}`);  
}
```



Use callback

```
let myPromise = new Promise(function(resolve, reject) {  
    setTimeout(function() {  
        resolve("world!");  
    }, 3000);  
});  
  
myPromise.then(function(value) {  
    console.log(`Hello ${value}`);  
});
```

JavaScript Async

*async & await
make promises
easier to write*



JavaScript Async

- Example

```
function printResult(result) {  
    console.log(result);  
}
```

```
async function sayHello() {  
    return "Hello";  
}
```

=

```
function sayHello() {  
    return Promise.resolve("Hello");  
}
```

```
sayHello().then(  
    function(value) {  
        printResult(value);  
    },  
    function(error) {  
        printResult(error);  
    }  
);
```

JavaScript Async

- Example

```
function printResult(result) {  
  console.log(result);  
}
```

```
async function sayHello() {  
  return "Hello";  
}
```

=

```
function sayHello() {  
  return Promise.resolve("Hello");  
}
```

```
var result = await sayHello();  
printResult(result);
```

JavaScript Async

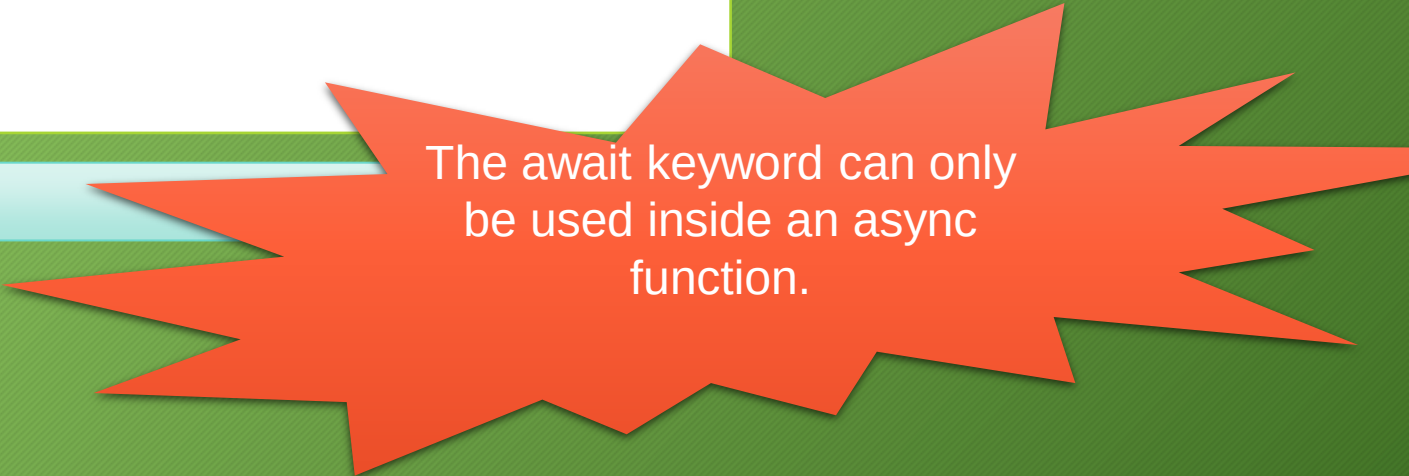
- The keyword **async** before a function makes the function return a promise
- The **await** keyword makes the function pause the execution and wait for a resolved promise before it continues

JavaScript Async

- Example

```
async function sayHello() {  
  let myPromise = new Promise(function(resolve) {  
    resolve("Hello");  
  });  
  
  var result = await myPromise;  
  console.log(result);  
}
```

```
sayHello();
```



The await keyword can only be used inside an async function.

Exercise 1

- Viết một chương trình mô phỏng quy trình **đặt hàng trực tuyến**, gồm các bước sau:
 1. **Chọn sản phẩm**: Người dùng chọn một sản phẩm (tốn 1 giây) => **Đã chọn sản phẩm!**
 2. **Thanh toán**: Sau khi chọn sản phẩm, tiến hành thanh toán (tốn 2 giây) => **Đã thanh toán thành công!**
 3. **Giao hàng**: Sau khi thanh toán xong, đơn hàng sẽ được giao (tốn 3 giây). → **Đơn hàng đã được giao!**
- Mỗi bước hoàn thành, chương trình sẽ gọi một **callback** để tiếp tục thực hiện bước tiếp theo.

Exercise 2

- Viết một chương trình giả lập quá trình lấy thông tin người dùng từ cơ sở dữ liệu và sau đó hiển thị thông tin đó.
- Yêu cầu:
 - Hàm **getUser(id, callback)**: mô phỏng việc lấy dữ liệu từ database (mất 2 giây).
 - Sau khi lấy được dữ liệu, gọi callback để hiển thị thông tin người dùng (**displayUser**).
 - Gọi **getUser()** hai lần để lấy thông tin của hai người khác nhau
- Cho trước hàm **displayUser**:

```
function displayUser(user) {  
    console.log(`Tên: ${user.name}, Tuổi: ${user.age}`);  
}
```


Exercise 3

- Sử dụng Promise
- Sử dụng Async / Await



Cho Exercise 1, 2



Your opinion

Can you give your opinion on Callback, Promise, and Async/Await?

Exercise 4

- Sử dụng Promise
- Sử dụng Async / Await



Để cải tiến module
db_model

Thank you!